

5.11 Template Method Pattern

Template Method Pattern เป็นแพตเทิร์นพฤติกรรมที่ใช้ในการกำหนดโครงสร้างลำดับขั้นตอนการทำงานในคลาสแม่ (Abstract Class) และอนุญาตให้คลาสลูก (Concrete Classes) สามารถกำหนดรายละเอียดเฉพาะของแต่ละขั้นตอน

:: *Template Method Pattern* ::

กำหนดโครงสร้างลำดับขั้นตอนการทำงานในคลาสแม่ และอนุญาตให้คลาสลูกสามารถกำหนดรายละเอียดเฉพาะของแต่ละขั้นตอน

เน้นการนำโค้ดกลับมาใช้ใหม่ (Code Reusability) และการควบคุมลำดับการดำเนินการ

ตัวอย่างปัญหา

บ้านอัจฉริยะมีหลายโหมดต่าง จึงมีระบบเตรียมบ้านสำหรับโหมดต่าง ๆ เช่น Night, Vacation, Party

ตัวอย่าง Main program

```
HomeMode partyMode = new PartyMode();  
partyMode.prepareHome();
```

5.11.1 วัตถุประสงค์และตัวอย่าง

Template Method Pattern กำหนดโครงสร้างหลักของอัลกอริทึมในเมธอดหนึ่งของคลาสแม่ (Superclass) ในขณะที่ให้คลาสลูก (Subclasses) มีความสามารถในการปรับเปลี่ยนขั้นตอนย่อยบางส่วนของอัลกอริทึมนั้นได้ เน้นการนำโค้ดกลับมาใช้ใหม่ (Code Reusability) และการควบคุมลำดับการดำเนินการ

- 1) การกำหนดโครงสร้างที่คงที่ (Define the Skeletal Structure) สร้างเมธอดหลัก (Template Method) ในคลาสแม่ เพื่อกำหนดลำดับที่แน่นอน และไม่เปลี่ยนแปลงของการดำเนินการ เมธอดแม่แบบจะเรียกใช้ชุดของเมธอดอื่น ๆ ตามลำดับที่กำหนดไว้ ซึ่งเมธอดเหล่านั้นบางส่วนจะเป็นเมธอดที่เป็นนามธรรม (Abstract) หรือเป็นเมธอดที่ถูกกำหนดค่าเริ่มต้นไว้ (Concrete Methods)

- 2) การอนุญาตให้ปรับเปลี่ยนได้เฉพาะส่วน (Allowing Selective Customization) เปิดโอกาสให้คลาสสามารถระบุรายละเอียดการทำงานของขั้นตอนย่อยบางขั้นตอนได้ โดยไม่ต้องเปลี่ยนโครงสร้างโดยรวมของอัลกอริทึม คลาสลูกจะ เขียนทับ (Override) เมธอดที่เป็นนามธรรมหรือเมธอดที่ต้องการปรับเปลี่ยน ซึ่งเมธอดเหล่านี้ถูกเรียกจาก Template Method ในคลาสแม่
- 3) การควบคุมการไหลของการดำเนินการ (Controlling the Flow) ควบคุมว่าขั้นตอนใดบ้างที่ต้องมีการดำเนินการ และในลำดับใด โดยป้องกันไม่ให้ Client หรือ Subclass เปลี่ยนลำดับหลัก Template Method มักจะถูกประกาศเป็น final เพื่อป้องกันไม่ให้คลาสลูกเขียนทับทั้ง Template Method ทำให้มั่นใจได้ว่าอัลกอริทึมหลักจะทำงานตามที่ออกแบบไว้เสมอ

ตัวอย่างขั้นตอนการสร้างบ้าน เมื่อ Template Method Pattern มีเมธอด `buildHouse()` ประกอบด้วยลำดับการสร้างบ้านดังนี้

- (1) วางรากฐาน (Concrete Step)
- (2) สร้างผนัง (Abstract Step) → ให้คลาสลูกกำหนดเอง
- (3) สร้างหลังคา (Abstract Step) → ให้คลาสลูกกำหนดเอง
- (4) ตกแต่งภายใน (Concrete Step)

คลาสลูกเช่น `WoodenHouse` และ `BrickHouse` จะทำ overriding ที่ขั้นตอน "สร้างผนัง" และ "สร้างหลังคา" เท่านั้นเนื่องจากบ้านไม้กับบ้านอิฐมีขั้นตอนต่างกัน ส่วนลำดับการสร้างบ้าน ได้แก่ วางรากฐาน → สร้างผนัง → สร้างหลังคา → ตกแต่ง ยังคงเดิมเสมอ

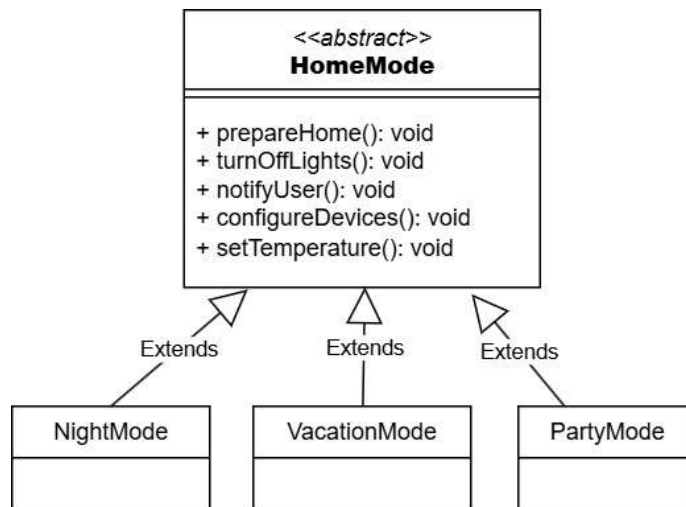
ตัวอย่างการใช้งาน

บ้านอัจฉริยะมีหลายโหมดต่าง จึงมีระบบเตรียมบ้านสำหรับโหมดต่าง ๆ เช่น

- Night Mode หมายถึง เตรียมบ้านสำหรับการนอน
- Vacation Mode หมายถึง เตรียมบ้านสำหรับการออกไปเที่ยว
- Party Mode หมายถึง เตรียมบ้านสำหรับการจัดงานเลี้ยง

แต่ละโหมดมีขั้นตอนที่คล้ายกัน เช่น ปิดไฟ เปิด/ปิดอุปกรณ์บางอย่าง และตั้งค่าอุณหภูมิ โครงสร้างระบบ

- Template Class กำหนดลำดับขั้นตอนการเตรียมบ้าน
- Concrete Classes กำหนดรายละเอียดเฉพาะสำหรับแต่ละโหมด



ภาพที่ xx ระบบบ้านอัจฉริยะที่มีหลายโหมดด้วย Template Method Pattern

การทำงานของระบบบ้านอัจฉริยะให้อยู่ในโหมดต่าง ๆ `prepareHome()` มีลำดับที่แน่นอนดังต่อไปนี้

- 1) ปิดไฟ (`turnOffLights()`)
- 2) ตั้งค่าอุปกรณ์ (`configureDevices()`)
- 3) ตั้งค่าอุณหภูมิ (`setTemperature()`)
- 4) แจ้งเตือนผู้ใช้ (`notifyUser()`)

HomeMode (Abstract Class)

- กำหนดโครงสร้างของขั้นตอนการเตรียมบ้านผ่านเมธอด `prepareHome()`
- มีเมธอดที่บังคับให้คลาสลูกต้องกำหนดเอง (`configureDevices()` และ `setTemperature()`)

```

1. // Abstract Template Class
2. abstract class HomeMode {
3.
4.     // Template Method
5.     public final void prepareHome() {
6.         turnOffLights();
7.         configureDevices();
8.         setTemperature();
9.         notifyUser();
10.    }
11.

```

```

12. // Steps with default implementation
13. protected void turnOffLights() {
14.     System.out.println("Turning off all lights...");
15. }
16.
17. // Steps to be implemented by subclasses
18. protected abstract void configureDevices();
19. protected abstract void setTemperature();
20.
21. // Hook method (optional step)
22. protected void notifyUser() {
23.     System.out.println("User has been notified.");
24. }
25.}

```

NightMode, VacationMode, PartyMode (**Concrete Classes**) กำหนดรายละเอียดเฉพาะของแต่ละโหมด เช่น

- Night Mode หมายถึง ล็อคประตูและหน้าต่าง
- Vacation Mode หมายถึง เปิดกล้องวงจรปิดและปิดเครื่องปรับอากาศ
- Party Mode หมายถึง เปิดไฟตกแต่งและระบบเสียง

```

1. // Concrete Class: Night Mode
2. class NightMode extends HomeMode {
3.
4.     @Override
5.     protected void configureDevices() {
6.         System.out.println("Locking all doors and windows...");
7.     }
8.
9.     @Override
10.    protected void setTemperature() {
11.        System.out.println("Setting temperature to 22°C
12.                            for comfortable sleep.");
13.    }
14.}

```

```

1. // Concrete Class: Vacation Mode
2. class VacationMode extends HomeMode {
3.
4.     @Override
5.     protected void configureDevices() {
6.         System.out.println("Activating security cameras
7.                             and alarm system...");
8.     }
9.
10.    @Override
11.    protected void setTemperature() {
12.        System.out.println("Turning off heating and cooling
13.                            systems to save energy.");
14.    }

```

```

15.
16.     @Override
17.     protected void notifyUser() {
18.         System.out.println("User notified via mobile app
19.                             about Vacation Mode activation.");
20.     }
21.}

```

```

1. // Concrete Class: Party Mode
2. class PartyMode extends HomeMode {
3.
4.     @Override
5.     protected void configureDevices() {
6.         System.out.println("Turning on decorative lights and
7.                             sound system...");
8.     }
9.
10.    @Override
11.    protected void setTemperature() {
12.        System.out.println("Setting temperature to 20°C
13.                            for a lively atmosphere.");
14.    }
15.}

```

ตัวอย่างการใช้งาน

```

1. // Main Class
2. public class TemplateMethodExample {
3.     public static void main(String[] args) {
4.
5.         System.out.println("\nPreparing home for Party Mode:");
6.         HomeMode partyMode = new PartyMode();
7.         partyMode.prepareHome();
8.     }
9. }

```

ผลลัพธ์

Preparing home for Party Mode:

Turning off all lights...

Turning on decorative lights and sound system...

Setting temperature to 20°C for a lively atmosphere.

User has been notified.

วัตถุประสงค์ในการใช้งาน Template Method pattern

- สร้างโครงสร้างขั้นตอนที่แน่นอน โดยควบคุมลำดับการทำงานของขั้นตอนต่าง ๆ ในกระบวนการ
- ปรับแต่งรายละเอียดในคลาสลูก โดยเปิดโอกาสให้คลาสลูกสามารถเปลี่ยนพฤติกรรมบางส่วนได้โดยไม่กระทบโครงสร้างโดยรวม
- ลดความซับซ้อนและเพิ่มการนำกลับมาใช้ใหม่ โค้ดที่ซ้ำซ้อนในกระบวนการเดียวกันจะถูกจัดการในคลาสแม่

ความจำเป็นในการใช้ Template Method Pattern

- ควบคุมลำดับการทำงาน ช่วยให้โครงสร้างการเตรียมบ้านมีลำดับขั้นตอนที่แน่นอน
- ความยืดหยุ่น แต่ละโหมดสามารถปรับแต่งรายละเอียดได้ตามความต้องการ โดยไม่ต้องเปลี่ยนโค้ดในคลาสแม่
- ลดการทำซ้ำ โค้ดส่วนที่เหมือนกัน เช่น การปิดไฟ ถูกกำหนดไว้ในคลาสแม่เพียงที่เดียว

5.11.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Template Method Pattern เหมาะสำหรับระบบที่มีขั้นตอนการทำงานแบบลำดับที่ชัดเจน และต้องการให้คลาสลูกสามารถปรับแต่งพฤติกรรมในบางส่วนได้ ช่วยลดการทำซ้ำของโค้ด และทำให้สามารถควบคุมลำดับของขั้นตอนในกระบวนการได้ง่ายขึ้น

ตัวอย่างที่ 1 ระบบประมวลผลการชำระเงิน (Payment Processing System) ระบบชำระเงินสามารถรองรับวิธีการชำระเงินที่แตกต่างกัน เช่น

- บัตรเครดิต
- PayPal
- โอนเงินผ่านธนาคาร

แต่ละวิธีมีขั้นตอนการทำงานคล้ายกัน เช่น การตรวจสอบยอดเงิน การชำระเงิน และการแจ้งผลลัพธ์

```
1. // Abstract Template Class
2. abstract class PaymentProcessor {
3.
4.     public final void processPayment(double amount) {
5.         validateAmount(amount);
6.         performPayment(amount);
7.         sendNotification();
8.     }
9. }
```

```

8.     }
9.
10.    protected void validateAmount(double amount) {
11.        System.out.println("Validating amount: $"
12.                            + amount);
13.    }
14.
15.    protected abstract void performPayment(
16.        double amount);
17.
18.    protected void sendNotification() {
19.        System.out.println("Notification:
20.                            Payment processed successfully.");
21.    }
22.}

```

```

1. // Concrete Class: Credit Card Payment
2. class CreditCardPayment extends PaymentProcessor {
3.     @Override
4.     protected void performPayment(double amount) {
5.         System.out.println("Processing credit card
6.                             payment of $" + amount);
7.     }

```

```

1. // Concrete Class: PayPal Payment
2. class PayPalPayment extends PaymentProcessor {
3.     @Override
4.     protected void performPayment(double amount) {
5.         System.out.println("Processing PayPal
6.                             payment of $" + amount);
7.     }
8. }

```

```

1. // Concrete Class: Bank Transfer
2. class BankTransferPayment extends PaymentProcessor {
3.     @Override
4.     protected void performPayment(double amount) {
5.         System.out.println("Processing bank transfer
6.                             payment of $" + amount);
7.     }
8. }

```

```

1. // Main Class
2. public class PaymentProcessingExample {
3.     public static void main(String[] args) {
4.
5.         PaymentProcessor creditCard = new
6.             CreditCardPayment();
7.         creditCard.processPayment(100.0);

```

```

8.
9.      System.out.println();
10.     PaymentProcessor paypal = new PayPalPayment();
11.     paypal.processPayment(200.0);
12.
13.     System.out.println();
14.     PaymentProcessor bankTransfer = new
15.         BankTransferPayment();
16.     bankTransfer.processPayment(300.0);
17. }
18.}

```

ผลลัพธ์

Validating amount: \$100.0

Processing credit card payment of \$100.0

Notification: Payment processed successfully.

Validating amount: \$200.0

Processing PayPal payment of \$200.0

Notification: Payment processed successfully.

Validating amount: \$300.0

Processing bank transfer payment of \$300.0

Notification: Payment processed successfully.

ตัวอย่างที่ 2 ระบบเตรียมอาหาร (Food Preparation System) ระบบเตรียมอาหารในร้าน

อาหาร มีเมนูต่าง ๆ เช่น พิซซ่า เบอร์เกอร์ สลัด แต่ละเมนูมีลำดับขั้นตอนคล้ายกัน เช่น การเตรียม ส่วนผสม การปรุงอาหาร และการเสิร์ฟ

```

1. // Abstract Template Class
2. abstract class FoodPreparation {
3.
4.     public final void prepareFood() {
5.         prepareIngredients();
6.         cook();
7.         serve();
8.     }
9.
10.    protected abstract void prepareIngredients();
11.    protected abstract void cook();
12.    protected void serve() {
13.        System.out.println("Serving the food.");
14.    }

```

15.}

```
1. // Concrete Class: Pizza
2. class Pizza extends FoodPreparation {
3.
4.     @Override
5.     protected void prepareIngredients() {
6.         System.out.println("Preparing dough, sauce
7.                             , and cheese.");
8.     }
9.
10.    @Override
11.    protected void cook() {
12.        System.out.println("Baking pizza in the oven.");
13.    }
14.}
```

```
1. // Concrete Class: Burger
2. class Burger extends FoodPreparation {
3.
4.     @Override
5.     protected void prepareIngredients() {
6.         System.out.println("Preparing buns, patties,
7.                             and vegetables.");
8.     }
9.
10.    @Override
11.    protected void cook() {
12.        System.out.println("Grilling the patties.");
13.    }
14.}
```

```
1. // Concrete Class: Salad
2. class Salad extends FoodPreparation {
3.     @Override
4.     protected void prepareIngredients() {
5.         System.out.println("Chopping vegetables and
6.                             preparing dressing.");
7.     }
8.
9.     @Override
10.    protected void cook() {
11.        System.out.println("Mixing the ingredients
12.                            (no cooking required).");
13.    }
14.}
```

```
1. // Main Class
2. public class FoodPreparationExample {
```

```

3.      public static void main(String[] args) {
4.
5.          FoodPreparation pizza = new Pizza();
6.          System.out.println("Preparing Pizza:");
7.          pizza.prepareFood();
8.
9.          System.out.println();
10.
11.         FoodPreparation burger = new Burger();
12.         System.out.println("Preparing Burger:");
13.         burger.prepareFood();
14.
15.         System.out.println();
16.
17.         FoodPreparation salad = new Salad();
18.         System.out.println("Preparing Salad:");
19.         salad.prepareFood();
20.     }
21. }

```

ผลลัพธ์

Preparing Pizza:

Preparing dough, sauce, and cheese.

Baking pizza in the oven.

Serving the food.

Preparing Burger:

Preparing buns, patties, and vegetables.

Grilling the patties.

Serving the food.

Preparing Salad:

Chopping vegetables and preparing dressing.

Mixing the ingredients (no cooking required).

Serving the food.

ตัวอย่างที่ 3 ระบบรายงานข้อมูล (Report Generation System) ในระบบสร้างรายงาน
สามารถสร้างรายงานได้หลากหลาย เช่น รายงานยอดขาย รายงานสินค้าคงคลัง รายงานพนักงาน

แต่ละรายงานมีโครงสร้างคล้ายกัน เช่น การรวบรวมข้อมูล การวิเคราะห์ และการสร้างไฟล์รายงาน

```
1. // Abstract Template Class
2. abstract class ReportGenerator {
3.     public final void generateReport() {
4.         collectData();
5.         analyzeData();
6.         exportReport();
7.     }
8.
9.     protected abstract void collectData();
10.    protected abstract void analyzeData();
11.
12.    protected void exportReport() {
13.        System.out.println("Exporting report to PDF.");
14.    }
15.}
```

```
1. // Concrete Class: Sales Report
2. class SalesReport extends ReportGenerator {
3.     @Override
4.     protected void collectData() {
5.         System.out.println("Collecting sales data.");
6.     }
7.
8.     @Override
9.     protected void analyzeData() {
10.        System.out.println("Analyzing sales trends.");
11.    }
12.}
```

```
1. // Concrete Class: Inventory Report
2. class InventoryReport extends ReportGenerator {
3.     @Override
4.     protected void collectData() {
5.         System.out.println("Collecting inventory data.");
6.     }
7.
8.     @Override
9.     protected void analyzeData() {
10.        System.out.println("Analyzing inventory
11.                            levels.");
12.    }
13.}
```

```
1. // Concrete Class: Employee Report
2. class EmployeeReport extends ReportGenerator {
3.
4.     @Override
5.     protected void collectData() {
```

```

6.         System.out.println("Collecting employee data.");
7.     }
8.
9.     @Override
10.    protected void analyzeData() {
11.        System.out.println("Analyzing employee
12.                             performance.");
13.    }
14.}

```

```

1. // Main Class
2. public class ReportGenerationExample {
3.     public static void main(String[] args) {
4.
5.         ReportGenerator salesReport = new SalesReport();
6.         System.out.println("Generating Sales Report:");
7.         salesReport.generateReport();
8.
9.         System.out.println();
10.
11.        ReportGenerator inventoryReport =
12.            new InventoryReport();
13.        System.out.println("Generating Inventory
14.                             Report:");
15.        inventoryReport.generateReport();
16.
17.        System.out.println();
18.
19.        ReportGenerator employeeReport = new
20.            EmployeeReport();
21.        System.out.println("Generating Employee
22.                             Report:");
23.        employeeReport.generateReport();
24.    }
25.}

```

ผลลัพธ์

Generating Sales Report:

Collecting sales data.

Analyzing sales trends.

Exporting report to PDF.

Generating Inventory Report:

Collecting inventory data.

Analyzing inventory levels.

Exporting report to PDF.

Generating Employee Report:
Collecting employee data.
Analyzing employee performance.
Exporting report to PDF.

5.12 Visitor Pattern

Visitor Pattern เป็นแพตเทิร์นพฤติกรรมที่แยกการดำเนินการ (operation) ออกจากโครงสร้างวัตถุ (object structure) เหมาะกับสถานการณ์ที่ต้องการเพิ่มฟังก์ชันการทำงานให้กับวัตถุโดยไม่ต้องแก้ไขคลาสของวัตถุเหล่านั้น

:: Visitor Pattern ::

แยกการดำเนินการ (operation) ออกจากโครงสร้างวัตถุ (object structure)
เหมาะกับสถานการณ์ที่ต้องการเพิ่มฟังก์ชันการทำงานให้กับวัตถุโดยไม่ต้องแก้ไขคลาสของวัตถุเหล่านั้น

ตัวอย่างปัญหา

ระบบบ้านอัจฉริยะ มีการตรวจสอบอุปกรณ์ในบ้าน เช่น ไฟ เครื่องปรับอากาศ กล้องวงจรปิด
ต้องการเพิ่มฟังก์ชันการตรวจสอบโดยไม่ต้องแก้ไขคลาสของอุปกรณ์

ตัวอย่าง Main program

House

5.12.1 วัตถุประสงค์และตัวอย่าง

Visitor Pattern แยกอัลกอริทึม (Operation) ออกจากโครงสร้างอ็อบเจกต์ (Structure) ที่ทำงานด้วย ช่วยให้สามารถเพิ่มฟังก์ชันการทำงานใหม่ให้กับคลาสที่มีอยู่แล้ว โดย ไม่ต้องแก้ไขคลาสเหล่านั้น ทำให้เพิ่มฟังก์ชันใหม่ได้ง่าย

- 1) การแยกการดำเนินการออกจากองค์ประกอบ (Decoupling Operation and Element) ถ้าต้องการเพิ่มฟังก์ชันใหม่ให้กับโครงสร้างอ็อบเจกต์ เช่น การคำนวณภาษี การบันทึกเป็น XML จะต้องแก้ไขโค้ดในทุกคลาสขององค์ประกอบในโครงสร้าง จึงควรใช้ Visitor Pattern ซึ่งประกอบด้วยคลาสดังนี้
 - Element Interface (องค์ประกอบ) มีเมธอดเดียวคือ accept(Visitor visitor)
 - Visitor Interface (ผู้เยี่ยมชม) มีเมธอด visit() สำหรับ Element แต่ละประเภท (visitConcreteElementA(), visitConcreteElementB())
- 2) การเพิ่มฟังก์ชันใหม่ได้ง่าย (Adding New Operations) เมื่อต้องการเพิ่มฟังก์ชันใหม่ เช่น การแสดงผลข้อมูลในรูปแบบ JSON เพียงสร้างคลาส ConcreteVisitor ใหม่ โดยไม่ต้องแก้ไขคลาสขององค์ประกอบเดิม
- 3) การรวบรวมตรรกะที่เกี่ยวข้องไว้ที่เดียว (Centralizing Related Logic) ตรรกะการประมวลผลที่เกี่ยวข้องกับหลายคลาสในโครงสร้างถูกรวมไว้ที่เดียว เช่น การพิมพ์ทั้งหมดถูกรวมไว้ในคลาส Visitor เพียงคลาสเดียว แทนที่จะกระจายไปในคลาส Element ต่าง ๆ

ตัวอย่างการใช้งาน

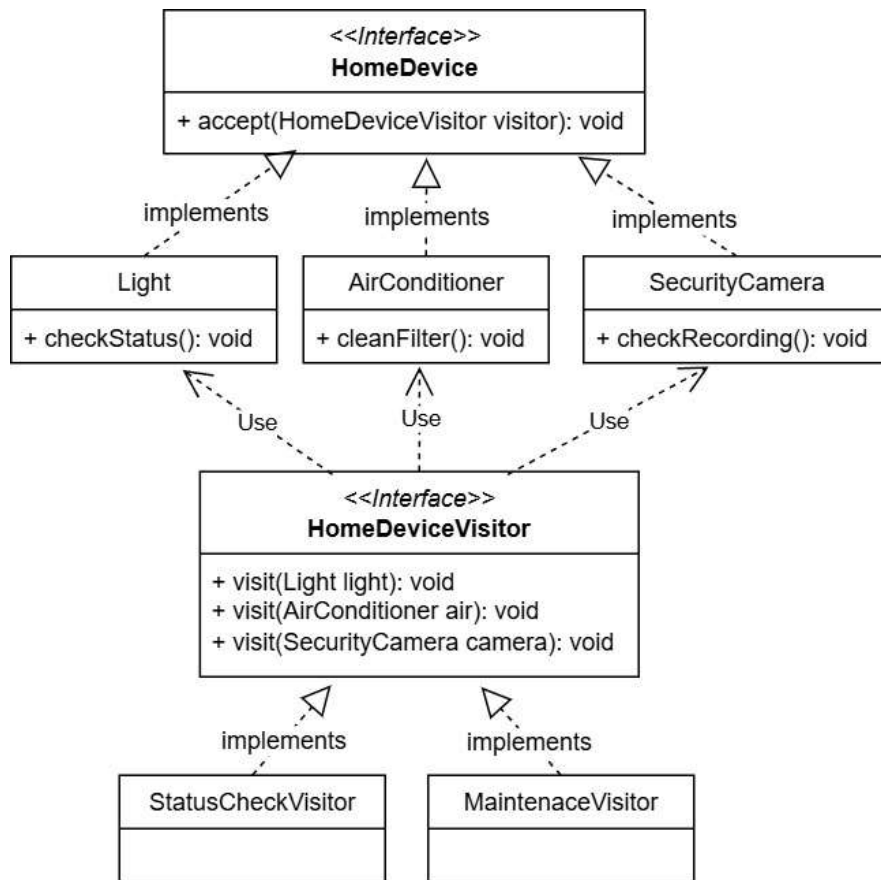
ระบบบ้านอัจฉริยะ มีการตรวจสอบและบำรุงรักษาอุปกรณ์ในบ้าน อุปกรณ์ต่าง ๆ เช่น

- ไฟ (Light) ต้องตรวจสอบว่าหลอดไฟทำงานได้ดีหรือไม่
- เครื่องปรับอากาศ (AirConditioner) ต้องตรวจสอบระดับความเย็น และทำความสะอาดตัวกรอง
- กล้องวงจรปิด (SecurityCamera) ต้องตรวจสอบภาพและการบันทึก

Visitor Pattern ช่วยให้สามารถเพิ่มฟังก์ชันการตรวจสอบ หรือบำรุงรักษาได้โดยไม่ต้องแก้ไขคลาสของอุปกรณ์

โครงสร้างโค้ด

- Element Interface อุปกรณ์ในบ้านต้องยอมรับ Visitor
- Concrete Elements เช่น ไฟ, เครื่องปรับอากาศ, กล้องวงจรปิด
- Visitor Interface กำหนดพฤติกรรมที่สามารถดำเนินการกับอุปกรณ์
- Concrete Visitors เช่น การตรวจสอบสถานะ การบำรุงรักษา



ภาพที่ xx ระบบบ้านอัจฉริยะที่แยกการควบคุมเพื่อทำหน้าที่ต่าง ๆ โดย Visitor Pattern

HomeDevice (Element Interface) กำหนดให้ทุกอุปกรณ์ในบ้านต้องสามารถรับ Visitor ผ่านเมธอด `accept(HomeDeviceVisitor visitor)`

```

1. // Element Interface
2. interface HomeDevice {
3.     void accept(HomeDeviceVisitor visitor);
4. }
  
```

Light, AirConditioner, SecurityCamera (Concrete Elements) อุปกรณ์แต่ละตัวกำหนดพฤติกรรมเฉพาะ เช่น ตรวจสอบสถานะหรือการทำความสะอาด

```

1. // Concrete Element: Light
2. class Light implements HomeDevice {
3.
4.     @Override
5.     public void accept(HomeDeviceVisitor visitor) {
6.         visitor.visit(this);
7.     }
8.
9.     public void checkStatus() {
10.        System.out.println("Light: Bulb is working fine.");
11.    }
  
```

```
12.}
```

```
1. // Concrete Element: AirConditioner
2. class AirConditioner implements HomeDevice {
3.
4.     @Override
5.     public void accept(HomeDeviceVisitor visitor) {
6.         visitor.visit(this);
7.     }
8.
9.     public void cleanFilter() {
10.        System.out.println("AirConditioner:
11.                               Filter has been cleaned.");
12.    }
13.}
```

```
1. // Concrete Element: Security Camera
2. class SecurityCamera implements HomeDevice {
3.
4.     @Override
5.     public void accept(HomeDeviceVisitor visitor) {
6.         visitor.visit(this);
7.     }
8.
9.     public void checkRecording() {
10.        System.out.println("SecurityCamera: Recording is
11.                               working fine.");
12.    }
13.}
```

```
1. // Visitor Interface
2. interface HomeDeviceVisitor {
3.     void visit(Light light);
4.     void visit(AirConditioner airConditioner);
5.     void visit(SecurityCamera securityCamera);
6. }
```

StatusCheckVisitor, MaintenanceVisitor (Concrete Visitors) กำหนดพฤติกรรมเฉพาะ เช่น การตรวจสอบสถานะ (StatusCheckVisitor) และการบำรุงรักษา (MaintenanceVisitor)

```
1. // Concrete Visitor: Status Check
2. class StatusCheckVisitor implements HomeDeviceVisitor {
3.
4.     @Override
5.     public void visit(Light light) {
6.         light.checkStatus();
7.     }
8. }
```

```

9.      @Override
10. public void visit(AirConditioner airConditioner) {
11.     System.out.println("AirConditioner: Status is normal.");
12. }
13.
14. @Override
15. public void visit(SecurityCamera securityCamera) {
16.     securityCamera.checkRecording();
17. }
18.}

```

HomeDeviceVisitor (Visitor Interface) กำหนดเมธอด visit() สำหรับอุปกรณ์แต่ละประเภท

```

1. // Concrete Visitor: Maintenance
2. class MaintenanceVisitor implements HomeDeviceVisitor {
3.
4.     @Override
5.     public void visit(Light light) {
6.         System.out.println("Light: No maintenance required.");
7.     }
8.
9.     @Override
10.    public void visit(AirConditioner airConditioner) {
11.        airConditioner.cleanFilter();
12.    }
13.
14.    @Override
15.    public void visit(SecurityCamera securityCamera) {
16.        System.out.println("SecurityCamera:
17.                               Lenses have been cleaned.");
18.    }
19.}

```

ตัวอย่างการใช้งาน สร้างอุปกรณ์ในบ้านและเรียก Visitor เพื่อตรวจสอบและบำรุงรักษา

```

1. // Main Class
2. public class VisitorPatternExample {
3.     public static void main(String[] args) {
4.
5.         HomeDevice[] devices = {
6.             new Light(),
7.             new AirConditioner(),
8.             new SecurityCamera()
9.         };
10.
11.        System.out.println("Performing Status Check:");
12.        HomeDeviceVisitor statusCheck =
13.            new StatusCheckVisitor();
14.        for (HomeDevice device : devices) {
15.            device.accept(statusCheck);
16.        }
17.
18.        System.out.println("\nPerforming Maintenance:");
19.        HomeDeviceVisitor maintenance =
20.            new MaintenanceVisitor();
21.        for (HomeDevice device : devices) {

```

```

22.         device.accept(maintenance);
23.     }
24. }
25.}

```

ผลลัพธ์

Performing Status Check:

Light: Bulb is working fine.

AirConditioner: Status is normal.

SecurityCamera: Recording is working fine.

Performing Maintenance:

Light: No maintenance required.

AirConditioner: Filter has been cleaned.

SecurityCamera: Lenses have been cleaned.

5.12.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Visitor Pattern เหมาะสำหรับระบบที่มีโครงสร้างวัตถุหลากหลายและต้องการเพิ่มฟังก์ชันการทำงานในอนาคต ช่วยให้เพิ่มฟังก์ชันการทำงานใหม่ได้ง่าย เช่น การตรวจสอบและการรายงาน โดยไม่กระทบกับโครงสร้างของอุปกรณ์

ตัวอย่างที่ 1 ระบบคำนวณค่าใช้จ่ายในบ้าน (Home Expense Calculation) ในบ้านอัจฉริยะ อุปกรณ์แต่ละประเภทอาจมีค่าใช้จ่ายด้านพลังงานแตกต่างกัน เช่น

- ไฟ (Light) คิดค่าไฟฟ้าตามชั่วโมงการใช้งาน
- เครื่องปรับอากาศ (AirConditioner) คิดค่าไฟฟ้าตามกำลังไฟและชั่วโมงการทำงาน
- เครื่องซักผ้า (WashingMachine) คิดค่าไฟฟ้าตามจำนวนรอบการทำงาน

Visitor Pattern จะช่วยให้การคำนวณค่าใช้จ่ายของอุปกรณ์ทั้งหมดทำได้ง่ายและแยกฟังก์ชันออกจากโครงสร้างของอุปกรณ์

```

1. // Element Interface
2. interface HomeDevice {
3.     void accept(HomeDeviceVisitor visitor);
4. }

```

```

1. // Concrete Element: Light
2. class Light implements HomeDevice {
3.
4.     private int hoursUsed;
5.
6.     public Light(int hoursUsed) {
7.         this.hoursUsed = hoursUsed;
8.     }
9.
10. public int getHoursUsed() {
11.     return hoursUsed;
12. }
13.
14. @Override
15. public void accept(HomeDeviceVisitor visitor) {
16.     visitor.visit(this);
17. }
18.}

```

```

1. // Concrete Element: AirConditioner
2. class AirConditioner implements HomeDevice {
3.
4.     private int hoursUsed;
5.     private int power; // in watts
6.
7.     public AirConditioner(int hoursUsed, int power) {
8.         this.hoursUsed = hoursUsed;
9.         this.power = power;
10. }
11.
12. public int getHoursUsed() {
13.     return hoursUsed;
14. }
15.
16. public int getPower() {
17.     return power;
18. }
19.
20. @Override
21. public void accept(HomeDeviceVisitor visitor) {
22.     visitor.visit(this);
23. }
24.}

```

```

1. // Concrete Element: WashingMachine
2. class WashingMachine implements HomeDevice {
3.
4.     private int cycles;
5.
6.     public WashingMachine(int cycles) {
7.         this.cycles = cycles;
8.     }
9.

```

```

10. public int getCycles() {
11.     return cycles;
12. }
13.
14. @Override
15. public void accept(HomeDeviceVisitor visitor) {
16.     visitor.visit(this);
17. }
18.}

```

```

1. // Visitor Interface
2. interface HomeDeviceVisitor {
3.     void visit(Light light);
4.     void visit(AirConditioner airConditioner);
5.     void visit(WashingMachine washingMachine);
6. }

```

```

1. // Concrete Visitor: Expense Calculator
2. class ExpenseCalculatorVisitor implements HomeDeviceVisitor {
3.
4.     private double totalExpense = 0;
5.
6.     @Override
7.     public void visit(Light light) {
8.         // $0.1 per hour
9.         double expense = light.getHoursUsed() * 0.1;
10.        totalExpense += expense;
11.        System.out.println("Light expense: $" + expense);
12.    }
13.
14.    @Override
15.    public void visit(AirConditioner airConditioner) {
16.        // $0.001 per watt-hour
17.        double expense = airConditioner.getHoursUsed()
18.            * airConditioner.getPower() * 0.001;
19.        totalExpense += expense;
20.        System.out.println("AirConditioner expense: $"
21.            + expense);
22.    }
23.
24.    @Override
25.    public void visit(WashingMachine washingMachine) {
26.        // $0.5 per cycle
27.        double expense = washingMachine.getCycles() * 0.5;
28.        totalExpense += expense;
29.        System.out.println("WashingMachine expense: $"
30.            + expense);
31.    }
32.
33.    public double getTotalExpense() {
34.        return totalExpense;
35.    }
36.}

```

```

1. // Main Class
2. public class HomeExpenseExample {
3.     public static void main(String[] args) {
4.
5.         HomeDevice[] devices = {
6.             new Light(10),
7.             new AirConditioner(5, 2000),
8.             new WashingMachine(3)
9.         };
10.
11.         ExpenseCalculatorVisitor calculator =
12.             new ExpenseCalculatorVisitor();
13.
14.         for (HomeDevice device : devices) {
15.             device.accept(calculator);
16.         }
17.
18.         System.out.println("\nTotal Expense: $"
19.             + calculator.getTotalExpense());
20.     }
21. }

```

ผลลัพธ์

Light expense: \$1.0

AirConditioner expense: \$10.0

WashingMachine expense: \$1.5

Total Expense: \$12.5

ตัวอย่างที่ 2 ระบบตรวจสอบความปลอดภัยในบ้าน (Home Security Check) ในบ้านอัจฉริยะ

อุปกรณ์ต่าง ๆ ต้องได้รับการตรวจสอบ เช่น

- กล้องวงจรปิด ตรวจสอบภาพและการบันทึก
- ระบบสัญญาณเตือนภัย ตรวจสอบการทำงานของเซ็นเซอร์
- ประตูอัจฉริยะ ตรวจสอบสถานะล็อก

```

1. // Element Interface
2. interface SecurityDevice {
3.     void accept(SecurityVisitor visitor);
4. }

```

```

1. // Concrete Element: Security Camera
2. class SecurityCamera implements SecurityDevice {
3.
4.     @Override
5.     public void accept(SecurityVisitor visitor) {
6.         visitor.visit(this);
7.     }
8.
9.     public void checkRecording() {
10.        System.out.println("SecurityCamera: Recording
11.            is active and functional.");
12.    }
13.}

```

```

1. // Concrete Element: Alarm System
2. class AlarmSystem implements SecurityDevice {
3.
4.     @Override
5.     public void accept(SecurityVisitor visitor) {
6.         visitor.visit(this);
7.     }
8.
9.     public void checkSensors() {
10.        System.out.println("AlarmSystem: All
11.            sensors are operational.");
12.    }
13.}

```

```

1. // Concrete Element: Smart Door
2. class SmartDoor implements SecurityDevice {
3.     @Override
4.     public void accept(SecurityVisitor visitor) {
5.         visitor.visit(this);
6.     }
7.
8.     public void checkLock() {
9.        System.out.println("SmartDoor: Lock is secured.");
10.    }
11.}

```

```

1. // Visitor Interface
2. interface SecurityVisitor {
3.     void visit(SecurityCamera camera);
4.     void visit(AlarmSystem alarm);
5.     void visit(SmartDoor door);
6. }

```

```

1. // Concrete Visitor: Security Check Visitor
2. class SecurityCheckVisitor implements SecurityVisitor {

```

```

3.
4.     @Override
5.     public void visit(SecurityCamera camera) {
6.         camera.checkRecording();
7.     }
8.     @Override
9.     public void visit(AlarmSystem alarm) {
10.        alarm.checkSensors();
11.    }
12. @Override
13. public void visit(SmartDoor door) {
14.     door.checkLock();
15. }
16.}

```

```

1. // Main Class
2. public class SecurityCheckExample {
3.     public static void main(String[] args) {
4.
5.         SecurityDevice[] devices = {
6.             new SecurityCamera(),
7.             new AlarmSystem(),
8.             new SmartDoor()
9.         };
10.
11.         SecurityVisitor securityCheck = new
12.             SecurityCheckVisitor();
13.
14.         System.out.println("Performing Security Check:");
15.         for (SecurityDevice device : devices) {
16.             device.accept(securityCheck);
17.         }
18.     }
19.}

```

ผลลัพธ์

Performing Security Check:

SecurityCamera: Recording is active and functional.

AlarmSystem: All sensors are operational.

SmartDoor: Lock is secured.

ตัวอย่างที่ 3 ซอฟต์แวร์เกมที่มีตัวละครหลากหลาย Visitor Pattern เหมาะกับเกมที่มีตัวละคร
หลากหลาย และต้องการเพิ่มฟีเจอร์ใหม่ ๆ ที่อาจเปลี่ยนแปลงในอนาคต ตัวละครหลากหลาย เช่น

- Warrior มีพลังโจมตีสูง

- Mage ใช้เวทย์มนตร์
- Healer พินฟูพลังชีวิต

การดำเนินการที่ต้องใช้กับตัวละครในเกมอาจรวมถึง

- การคำนวณพลังโจมตี
- การใช้ไอเท็มพิเศษ
- การบันทึกสถิติ

Visitor Pattern ช่วยเพิ่มฟังก์ชันการทำงานใหม่โดยไม่ต้องแก้ไขคลาสของตัวละคร เช่น เพิ่มการสนับสนุนฟีเจอร์ "Apply Buff" หรือ "Calculate Damage" ให้กับตัวละคร

โครงสร้างโค้ด

- Element Interface หมายถึง ตัวละครในเกมที่รองรับ Visitor
- Concrete Elements หมายถึง ตัวละคร เช่น Warrior, Mage, Healer
- Visitor Interface หมายถึง ฟังก์ชันที่ใช้กับตัวละคร เช่น คำนวณพลังโจมตี
- Concrete Visitors เช่น การคำนวณพลังโจมตี และการใช้ไอเท็มพิเศษ

GameCharacter (Element Interface) ตัวละครในเกมรองรับเมธอด accept เพื่อให้ Visitor สามารถดำเนินการได้

```
1. // Element Interface
2. interface GameCharacter {
3.     void accept(CharacterVisitor visitor);
4. }
```

Warrior, Mage, Healer (Concrete Elements) ตัวละครแต่ละประเภทมีพฤติกรรมเฉพาะ เช่น พลังโจมตีหรือเวทย์มนตร์

```
1. // Concrete Element: Warrior
2. class Warrior implements GameCharacter {
3.
4.     private int attackPower = 50;
5.
6.     public int getAttackPower() {
7.         return attackPower;
8.     }
9.
10.    @Override
11.    public void accept(CharacterVisitor visitor) {
12.        visitor.visit(this);
13.    }
```

```
14.}
```

```
1. // Concrete Element: Mage
2. class Mage implements GameCharacter {
3.
4.     private int magicPower = 70;
5.
6.     public int getMagicPower() {
7.         return magicPower;
8.     }
9.
10.    @Override
11.    public void accept(CharacterVisitor visitor) {
12.        visitor.visit(this);
13.    }
14.}
```

```
1. // Concrete Element: Healer
2. class Healer implements GameCharacter {
3.
4.     private int healingPower = 30;
5.
6.     public int getHealingPower() {
7.         return healingPower;
8.     }
9.
10.    @Override
11.    public void accept(CharacterVisitor visitor) {
12.        visitor.visit(this);
13.    }
14.}
```

CharacterVisitor (Visitor Interface) กำหนดฟังก์ชันที่จะทำงานกับตัวละคร เช่น การคำนวณพลังโจมตี visit(Warrior), visit(Mage)

```
1. // Visitor Interface
2. interface CharacterVisitor {
3.     void visit(Warrior warrior);
4.     void visit(Mage mage);
5.     void visit(Healer healer);
6. }
```

DamageCalculatorVisitor, BuffApplierVisitor (Concrete Visitors) กำหนดฟังก์ชันเฉพาะ เช่น คำนวณพลังโจมตีและเพิ่มบัฟให้กับตัวละคร

```
1. // Concrete Visitor: Damage Calculator
2. class DamageCalculatorVisitor implements CharacterVisitor {
3.
4.     @Override
5.     public void visit(Warrior warrior) {
```

```

6.         System.out.println("Warrior deals damage: "
7.             + warrior.getAttackPower());
8.     }
9.
10.    @Override
11.    public void visit(Mage mage) {
12.        System.out.println("Mage deals damage: "
13.            + mage.getMagicPower());
14.    }
15.
16.    @Override
17.    public void visit(Healer healer) {
18.        System.out.println("Healer deals no
19.            significant damage.");
20.    }
21.}

```

```

1. // Concrete Visitor: Buff Applier
2. class BuffApplierVisitor implements CharacterVisitor {
3.
4.     @Override
5.     public void visit(Warrior warrior) {
6.         System.out.println("Warrior receives a +10 attack
7.             power buff.");
8.     }
9.
10.    @Override
11.    public void visit(Mage mage) {
12.        System.out.println("Mage receives a +20 magic
13.            power buff.");
14.    }
15.    @Override
16.    public void visit(Healer healer) {
17.        System.out.println("Healer receives a +10 healing
18.            power buff.");
19.    }
20.}

```

VisitorPatternGameExample (Main Class) สร้างตัวละครและเรียก Visitor เพื่อเพิ่มพฤติกรรมใหม่โดยไม่แก้ไขคลาสของตัวละคร

```

1. // Main Class
2. public class VisitorPatternGameExample {
3.     public static void main(String[] args) {
4.
5.         GameCharacter[] characters = {
6.             new Warrior(),
7.             new Mage(),
8.             new Healer()
9.         };
10.
11.        System.out.println("Calculating Damage:");
12.        CharacterVisitor damageCalculator = new
13.            DamageCalculatorVisitor();

```

```

14.     for (GameCharacter character : characters) {
15.         character.accept(damageCalculator);
16.     }
17.
18.     System.out.println("\nApplying Buffs:");
19.     CharacterVisitor buffApplier = new
20.         BuffApplierVisitor();
21.     for (GameCharacter character : characters) {
22.         character.accept(buffApplier);
23.     }
24. }
25.}

```

ผลลัพธ์

Calculating Damage:

Warrior deals damage: 50

Mage deals damage: 70

Healer deals no significant damage.

Applying Buffs:

Warrior receives a +10 attack power buff.

Mage receives a +20 magic power buff.

Healer receives a +10 healing power buff.

จะเห็นว่าการใช้ Visitor Pattern ในซอฟต์แวร์เกม ช่วยให้สนับสนุนการทำงานดังนี้

- เพิ่มพฤติกรรมใหม่ได้ง่าย เช่น เพิ่มระบบคำนวณคะแนนหรือความเสียหายพิเศษ โดยไม่ต้องแก้ไขตัวละคร
- รองรับตัวละครหลายประเภท เช่น Warrior, Mage, Healer ที่มีพฤติกรรมเฉพาะ
- ลดความซับซ้อนในโค้ด ทำให้โค้ดที่เกี่ยวข้องกับการคำนวณแยกออกจากโครงสร้างตัวละคร

5.13 บทสรุป

แพตเทิร์นพฤติกรรม (Behavioral Design Patterns) เป็นหนึ่งในสามกลุ่มหลักของ Design Patterns ถูกอธิบายไว้ในหนังสือ "Design Patterns: Elements of Reusable Object-Oriented Software" โดย Gang of Four (GoF) แพตเทิร์นกลุ่มนี้มุ่งเน้นไปที่การจัดการพฤติกรรมของ อ็อบเจกต์ (Objects) และวิธีการทำงานร่วมกันของวัตถุในระบบเพื่อให้โค้ดอ่านง่าย ยืดหยุ่น และเพิ่ม

ฟังก์ชันการทำงานได้โดยง่าย มีวัตถุประสงค์หลักคือ เพิ่มความยืดหยุ่นโดยแยกพฤติกรรมหรือการทำงานออกจากโครงสร้างหลักของวัตถุ ทำให้สามารถเปลี่ยนแปลงหรือเพิ่มฟังก์ชันใหม่ได้โดยไม่กระทบโค้ดเดิม ช่วยลดความซับซ้อนโดยลดการใช้เงื่อนไขซับซ้อน เช่น if-else หรือ switch-case ด้วยการนำหลัก Polymorphism มาใช้ ปรับปรุงการทำงานร่วมกันของวัตถุโดยจัดการการสื่อสารและความสัมพันธ์ระหว่างวัตถุอย่างชัดเจน รองรับการขยายระบบด้วยการเพิ่มความสามารถในการขยายโค้ดโดยไม่ต้องแก้ไขโครงสร้างหลัก และเพิ่มความสามารถในการนำกลับมาใช้ใหม่เพื่อให้โค้ดสามารถใช้งานได้หลายบริบท

Behavioral Design Patterns มีหลายรูปแบบ แต่ละรูปแบบถูกออกแบบมาเพื่อแก้ปัญหาต่าง ๆ ที่เกี่ยวข้องกับพฤติกรรมของวัตถุในระบบ

- 1) Strategy Pattern ใช้เมื่อต้องการสลับหรือเลือกใช้อัลกอริธึมได้อย่างยืดหยุ่นในขณะรันไทม์ มีวัตถุประสงค์เพื่อแยกอัลกอริธึมออกจากโครงสร้างหลักของระบบ ตัวอย่างเช่น ระบบบ้านอัจฉริยะที่เปลี่ยนโหมดการทำความร้อนได้ เช่น Eco Mode, Normal Mode, Turbo Mode
- 2) Template Method Pattern ใช้ในสถานการณ์ที่ต้องการกำหนดโครงสร้างขั้นตอนที่แน่นอน แต่อนุญาตให้คลาสลูกสามารถกำหนดรายละเอียดบางส่วนได้ มีวัตถุประสงค์เพื่อจัดลำดับการทำงานของขั้นตอนต่าง ๆ ในกระบวนการ และลดการทำซ้ำของโค้ด ตัวอย่างเช่น ระบบเตรียมบ้านสำหรับโหมดต่าง ๆ เช่น Night Mode, Vacation Mode, Party Mode
- 3) Observer Pattern ใช้เมื่อวัตถุหนึ่งต้องการแจ้งเตือนการเปลี่ยนแปลงไปยังวัตถุอื่น ๆ แบบเรียลไทม์ มีวัตถุประสงค์เพื่อลดการพึ่งพากันโดยตรง (loose coupling) ระหว่างวัตถุ ตัวอย่างเช่น ระบบบ้านอัจฉริยะที่เมื่อประตูเปิด อุปกรณ์อื่น ๆ เช่น ไฟและกล้องวงจรปิด จะเปิดทำงานโดยอัตโนมัติ
- 4) State Pattern ใช้ในสถานการณ์ที่วัตถุเปลี่ยนพฤติกรรมได้ตามสถานะของวัตถุ มีวัตถุประสงค์เพื่อแยกการจัดการสถานะออกจากวัตถุหลัก และลดการใช้ if-else ในการเช็คสถานะ ตัวอย่างเช่น ระบบควบคุมไฟในบ้านที่มีสถานะ Off, On, และ Dim
- 5) Visitor Pattern ใช้เมื่อวัตถุมีโครงสร้างที่แน่นอน แต่ต้องการเพิ่มฟังก์ชันใหม่ ๆ โดยไม่แก้ไขคลาสของวัตถุเหล่านั้น มีวัตถุประสงค์เพื่อแยกการดำเนินการออกจากโครงสร้างวัตถุ ตัวอย่างเช่น การตรวจสอบและบำรุงรักษาอุปกรณ์ในบ้าน เช่น ตรวจสอบสถานะไฟ กล้องวงจรปิด และเครื่องปรับอากาศ

- 6) Mediator Pattern ใช้เมื่อมีวัตถุหลายตัวที่ต้องสื่อสารกัน และต้องการลดความซับซ้อนของการเชื่อมโยงระหว่างวัตถุเหล่านั้น มีวัตถุประสงค์เพื่อใช้ตัวกลาง (Mediator) เป็นผู้จัดการการสื่อสาร ตัวอย่างเช่น ระบบบ้านอัจฉริยะที่ประสานงานระหว่างอุปกรณ์ต่าง ๆ เช่น เมื่อประตูเปิด ไฟเปิด และเครื่องปรับอากาศเริ่มทำงาน
- 7) Command Pattern ใช้เมื่อจำเป็นต้องบันทึกคำสั่งหรือการกระทำ เพื่อสามารถเรียกใช้หรือยกเลิกการกระทำนั้นได้ในอนาคต มีวัตถุประสงค์เพื่อแคปซูลคำสั่งให้เป็นออบเจกต์เดี่ยว ตัวอย่างเช่น ระบบรีโมทคอนโทรลในบ้านที่สามารถบันทึกคำสั่งเปิด/ปิดไฟหรือเครื่องปรับอากาศ
- 8) Chain of Responsibility Pattern ใช้ในสถานการณ์ที่คำร้องขอ (request) ต้องถูกส่งผ่านลำดับชั้นของผู้รับ (handlers) โดยไม่ต้องกำหนดผู้รับที่แน่นอนล่วงหน้า มีวัตถุประสงค์เพื่อส่งคำร้องขอผ่านโซ่ของออบเจกต์จนกว่าจะมีผู้จัดการคำร้องนั้น และลดการพึ่งพาระหว่างผู้ส่งคำร้องและผู้รับ ตัวอย่างเช่น ระบบบ้านอัจฉริยะที่ส่งคำสั่งเปิดไฟผ่านอุปกรณ์หลายตัว เช่น ฮับกลาง, ตัวควบคุมไฟ, หรืออุปกรณ์สำรอง
- 9) Interpreter Pattern ใช้ในสถานการณ์ที่ระบบต้องแปลความหมายหรือประเมินผลของคำสั่งที่ถูกกำหนดไว้ในรูปแบบไวยากรณ์ (grammar) มีวัตถุประสงค์เพื่อสร้างตัวแปลความหมาย (Interpreter) สำหรับภาษาเฉพาะในระบบ ทำให้ง่ายต่อการวิเคราะห์และประมวลผลคำสั่งหรือข้อมูลที่มีโครงสร้าง ตัวอย่างเช่น บ้านอัจฉริยะที่รองรับการแปลคำสั่งเสียง เช่น "เปิดไฟในห้องนั่งเล่น" และ "ลดอุณหภูมิเป็น 25°C"
- 10) Iterator Pattern ใช้ในสถานการณ์ที่ต้องการเข้าถึงองค์ประกอบ (elements) ของคอลเลกชัน (collection) อย่างเป็นลำดับ โดยไม่ต้องเปิดเผยโครงสร้างภายในของคอลเลกชัน มีวัตถุประสงค์เพื่อแยกความรับผิดชอบในการนำทาง (traverse) คอลเลกชันออกจากโครงสร้างคอลเลกชันเอง และสนับสนุนการวนรูปแบบหลายประเภท เช่น ลำดับไปข้างหน้า ย้อนกลับ หรือข้ามบางส่วน ตัวอย่างเช่น บ้านอัจฉริยะที่วนลูปผ่านอุปกรณ์ทั้งหมดเพื่อตรวจสอบสถานะ เช่น ไฟ, กล้องวงจรปิด, และเซ็นเซอร์
- 11) Memento Pattern ใช้ในสถานการณ์ที่ต้องการบันทึกและย้อนคืนสถานะของวัตถุ โดยไม่ละเมิดหลักการการห่อหุ้ม (Encapsulation) มีวัตถุประสงค์เพื่อบันทึกสถานะของวัตถุในช่วงเวลาใดเวลาหนึ่ง และสามารถย้อนคืนสถานะเดิมได้ในภายหลัง และช่วยลดความซับซ้อนในการจัดการการเปลี่ยนแปลงของวัตถุ ตัวอย่างเช่น ระบบบ้านอัจฉริยะที่สามารถบันทึกการตั้งค่าของไฟ, อุณหภูมิ, และโหมดบ้าน และสามารถย้อนกลับไปยังการตั้งค่าก่อนหน้าได้เมื่อผู้ใช้งานต้องการ

5.14 คำถามท้ายบท

- 1) Behavioral Design Patterns คืออะไร และแตกต่างจาก Structural และ Creational Patterns อย่างไร?
- 2) Strategy Pattern ช่วยแก้ปัญหาอะไร และมีโครงสร้างหลักอะไรบ้าง?
- 3) ใน Observer Pattern ใครคือ Subject และใครคือ Observer?
- 4) State Pattern เหมาะสำหรับสถานการณ์ใด?
- 5) Template Method Pattern ต่างจาก Strategy Pattern อย่างไร?
- 6) Visitor Pattern มีข้อดีและข้อเสียอย่างไร?
- 7) ใน Chain of Responsibility Pattern, โซ่ของตัวรับ (Chain of Handlers) มีวัตถุประสงค์อย่างไร?
- 8) อธิบายการทำงานของ Iterator Pattern
- 9) Interpreter Pattern เหมาะสำหรับสถานการณ์ใด?
- 10) Command Pattern ช่วยแก้ปัญหาอะไรในระบบซอฟต์แวร์?
- 11) Memento Pattern ช่วยอะไรในระบบที่มีการเปลี่ยนแปลงสถานะ?
- 12) ข้อแตกต่างหลักระหว่าง Observer และ Mediator Pattern คืออะไร?
- 13) อะไรคือหลักการสำคัญที่ State Pattern ใช้แก้ปัญหาการจัดการสถานะ?
- 14) ในระบบเกม, Visitor Pattern สามารถนำไปใช้กับตัวละครได้อย่างไร?
- 15) ระบุแพตเทิร์นที่เหมาะสมกับสถานการณ์นี้: "ระบบบ้านอัจฉริยะที่ต้องการส่งคำสั่งจากผู้ไปยังอุปกรณ์หลายตัวในลำดับที่แน่นอน"

5.15 เอกสารอ้างอิง

- 1) Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional.
- 2) Freeman, E., & Freeman, E. (2004). Head First Design Patterns: A Brain-Friendly Guide. O'Reilly Media.
- 3) Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley Professional.
- 4) Larman, C. (2004). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development. Prentice Hall.
- 5) Shalloway, A., & Trott, J. R. (2004). Design Patterns Explained: A New Perspective on Object-Oriented Design. Addison-Wesley Professional.

- 6) Buschmann, F., Henney, K., & Schmidt, D. C. (2007).
- 7) Pattern-Oriented Software Architecture: On Patterns and Pattern Languages. Wiley. Geary, D. M., & Horstmann, C. S. (2005). Core JavaServer Faces. Prentice Hall.
- 8) Alur, D., Malks, D., & Crupi, J. (2003). Core J2EE Patterns: Best Practices and Design Strategies. Prentice Hall.
- 9) Reenskaug, T. (1979). Model-View-Controller (MVC) Pattern. Xerox PARC.
- 10) Gamma, E. (1997). Design Patterns in Smalltalk MVC.